

Uncertainty Signals for Handwritten Math: Initial Results

Qwen2.5-VL-3B on FERMAT

The question

Can a model tell us when it is wrong, without an answer key? We test one idea: sample the model’s answer $K = 5$ times and measure how much it disagrees with itself (Shannon entropy over the clustered answers). If disagreement tracks error, it is a free error detector at deployment time. We test this on two tasks: **transcribing** a handwritten math solution, and **grading** whether that solution contains a mistake.

Setup

Qwen2.5-VL-3B on FERMAT. $K = 5$ samples at temperature 0.7 per prompt, plus 2 greedy samples as a determinism check. Entropy is computed only from the model’s own outputs; correctness is scored separately against ground truth. AUROC measures whether the two line up — concretely, how often a randomly chosen wrong item has higher entropy than a randomly chosen correct one. Intervals are 95% bootstrap CIs (10,000 resamples). $n = 300$ items, sampled to a balanced 150/150 split on whether the solution contains an error, with thresholds registered before the run.

Results

Task	AUROC (95% CI)	Model accuracy	Verdict
Transcription	0.835 [0.787, 0.879]	0.393	entropy predicts error
Grading	0.522 [0.462, 0.582]	0.517 (chance = 0.500)	no signal

Transcription: the signal is real (**works**)

When the model cannot read the handwriting it genuinely wavers, and that wavering tracks its errors. The result survives every check applied: removing all parse failures and all maximum-entropy items simultaneously still leaves AUROC = 0.796 [0.736, 0.852] at $n = 206$. It replicates an earlier $n = 100$ pilot (0.788) on a like-for-like comparison.

One directly usable rule falls out of it: **when all five transcriptions disagree, the model is wrong 57 times out of 61 (93.4% precision)**, catching 31% of all transcription errors. That is an abstention rule requiring no ground truth.

Grading: no signal, because the model cannot do the task (**fails**)

The reason matters more than the number. FERMAT is 84.8% error-containing. On a sample inheriting that imbalance the model scored 75% — which looks reasonable until you notice that always answering “there is an error,” without looking at anything, scores 87%. The model was *below* the do-nothing baseline. On a balanced set the flattery disappears:

Grading accuracy (balanced)	0.517	(chance = 0.500)
on solutions that do contain an error		0.947
on solutions that are correct		0.087
Answers “there is an error”		93% of all items

The model has one firm opinion — that everything contains a mistake — and repeats it, so it is perfectly self-consistent almost everywhere: 42% of items had all five samples identical, and only three distinct entropy values occurred across the entire run. There is nothing for entropy to measure.

What this means

Entropy measures *self-consistency*, never correctness. The two are separate quantities, and the method only works when they happen to align — i.e. when a model is steady while right and wavers while wrong. That holds for transcription here. It fails for grading, because a confidently wrong model is indistinguishable from a confidently right one: both look steady.

The clearest evidence is a sub-result: on the correct solutions, grading AUROC is **0.239** — below 0.5, meaning entropy ran *backwards*. The model was at its most consistent exactly when it was wrong, insisting on errors that were not there. Low entropy there is actively misleading, not merely uninformative.

So the headline is not “entropy does not work for reasoning.” It is **sampling-based uncertainty inherits the model’s competence**: it can flag errors for a model that knows something, and tells you nothing about a model that is guessing. Establishing that required rebalancing the evaluation set; on the original imbalanced data the problem was invisible.

Next step

The grading conclusion is about *this model*, not the method. The natural test is to repeat the grading arm with a stronger model (7B, or a text-only model given the ground-truth transcription) — gated on a capability check first: measure grading accuracy against the 0.500 baseline *before* computing any uncertainty metric. If it still grades at chance, the negative result stands for this task family; if it clears roughly 0.65–0.70, the entropy question becomes answerable and the existing analysis applies unchanged.

The transcription result is deployable as-is and deserves an operating-point analysis (precision/recall across thresholds, conformal calibration for guaranteed coverage). That needs no new model runs.

Full methodology, the two failed approaches that preceded the working transcription metric, the $n = 100$ pilot, and the diagnostic work behind the grading result are in `report/report.pdf`. All analysis code is unit tested in `pilot/`.